```
import * as React from "react"
import Venue from "./venue"
import {
  fetchData,
  initSubscription,
  venueStore,
} from "../contexts/venue_context"
import { Provider } from "react-redux"

export interface AppProps {
  concertId: number
  rowCount: number
  seatsPerRow: number
}

export const App = (props: AppProps): React.ReactElement => {
  venueStore.dispatch({ type: "initFromProps", props })
  initSubscription()
  venueStore.dispatch(fetchData())

  return (
    <Provider store={venueStore}>
      <Venue />
    </Provider>
  )
}

export default App
```

We're now doing two dispatches to set the initial state: one for the props, and one for the data fetch. They could be combined, but it seemed clearer to separate them. We're also calling `initSubscription()` to set up our subscription. The dispatch calls are just like `venueStore.dispatch(fetchData())`, using the thunk function just as if it were an action type.

Next up, we have two actions on the page that both call the dispatcher and notify the subscription—the on-click methods for clicking a seat and the Clear All buttons. We can turn both of those into thunk actions as well:

chapter_12/02/app/javascript/contexts/venue_context.ts

```
export const seatChange = (
```